

1. User Stories with Acceptance Criteria

Each story uses Given / When / Then acceptance criteria so the criterion is testable.

Story US-1 — Create a text note (FR-1)

As an engineer-user, I want to create a text note with a title and a markdown body, so that I can capture my thoughts in a familiar format.

- G: the application is open, W: I create a new note with title "Spike notes" and a markdown body, T: the note is persisted with created_at and updated_at set to now.
- G: I have created a note, W: I close and reopen the application, T: the note appears in my collection.
- G: I save a note with an empty title, T: the system rejects the save with a clear error.

Story US-2 — Mark a note as private (FR-4, SEC-1)

As a privacy-conscious user, I want to mark a note as private, so that its body is encrypted at rest and cannot be read without my passphrase.

- G: I have a note, W: I mark it private and set a passphrase, T: the body on disk is ciphertext.
- G: a SecureNote exists, W: I unlock it with the correct passphrase, T: the plaintext body is shown.
- G: I enter a wrong passphrase 3 times in a row, T: the system enforces a backoff and writes an audit-log entry.

Story US-3 — Search across notes (FR-8, NFR-1)

As a user with thousands of notes, I want to search by title or body text, so that I can find a note quickly.

- G: I have a 10,000-note collection, W: I issue a search query, T: results return in under 100 ms at p95.
- G: matching notes exist, T: results are ranked by recency.
- G: my search matches a SecureNote, T: only the title is shown in results until I unlock it.

Story US-4 — View and restore version history (FR-6)

As a user, I want to see prior versions of a note and restore one, so that I do not lose work after a bad edit.

- G: I have edited a note three times, W: I open its history, T: three prior versions are listed with timestamps.
- G: I select a prior version, W: I click restore, T: the current note becomes that prior version and a new history entry is appended.

Story US-5 — Cloud Sync across devices (FR-7)

As a user with two devices, I want my notes to sync, so that the same notes are available on both.

- G: I have notes on Device A, W: I enable sync and sign in on Device B, T: the same notes appear on Device B within 30 seconds.
- G: I edit the same note on A and B while offline, W: both devices come back online, T: a conflict is detected and the user is prompted to choose.
- G: sync runs, T: SecureNotes remain encrypted in transit and at rest on the server.

Story US-6 — Install a plugin without modifying core (Charter; supports FR-1/FR-4/FR-2)

As a user, I want to add a Voice-note plugin, so that I can capture audio notes without rewriting the app.

- G: the Voice plugin file is placed in the plugins directory, W: I restart, T: "Voice" appears as a note kind in the new-note menu.
- G: the plugin raises an error during load, T: the application starts normally without it and surfaces a non-fatal warning.

2. Prioritized Backlog (MoSCoW)

Rank	ID	Story	Requirements	Priority	Notes
1	US-1	Create a text note	FR-1, FR-5, NFR-2	Must	Foundational; everything else assumes a Note exists.
2	US-2	Mark a note as private	FR-4, SEC-1, SEC-2, SEC-4	Must	Defines the SecureNote/Encryption boundary the rest of the design depends on.
3	US-4	Version history	FR-6, SEC-4	Must	Lab 1 vague request "history of changes" — without this, the project doesn't satisfy the Charter.
4	US-3	Search across notes	FR-8, NFR-1	Should	Performance target requires a Sprint-Zero spike on indexing.
5	US-6	Plugin loading	Charter (Text/Voice/Sec)	Should	Architecturally important; functional content can come later.
6	US-5	Cloud Sync across devices	FR-7, SEC-1, SEC-4	Could	Largest unknown; sequenced after the local-first baseline is stable.

3. Sprint Zero Plan

Goal of Sprint Zero: make the team able to deliver US-1 through US-6 reliably without redoing setup work mid-flight.

Setup

- S0-1. Create the project repository with the layout `astranotes/{models,services,repository,view,plugins,tests}` to enforce NFR-2 at the directory level.
- S0-2. Pin Python version and create a `requirements.txt` with cryptography only, justified per SEC-3.
- S0-3. Configure pre-commit hooks for ruff, black, and mypy so style and type drift are caught before review.

Workflow readiness

- S0-4. Stand up the kanban board defined in working agreement.
- S0-5. Place the Definition of Done at the top of the repository as DOD so every PR explicitly checks it.
- S0-6. Decide on the prompt log format.

Planning artifacts

- S0-7. Confirm requirement IDs are the canonical IDs and every other artifact references them.
- S0-8. Draft skeleton UML so Sprint 1 has a target design instead of inventing one as it goes.

Technical decisions

- S0-9. Confirm SQLite as the default NoteRepository backing store; document the decision in the Architecture Decision Log.
- S0-10. Confirm `cryptography.fernet` as EncryptionService's primitive for SEC-1.

Risk reduction

- S0-11. Performance spike. Generate 10,000 synthetic notes and verify that title/body search returns under 100 ms p95 against a SQLite FTS5 index. If it does not, NFR-1 needs to be revisited before US-3 starts.
- S0-12. Encryption spike. Round-trip a SecureNote through `save` → `close` → `reopen` → `unlock` and confirm with a hex viewer that the body on disk is ciphertext. Lock the test in.
- S0-13. Sync risk register. Write down the three failure modes for FR-7 and decide which Sprint will address each.

Other Risk Reduction

- S0-14. Establish the rule that no change touches the encryption module without me reading the diff line-by-line.